Exercise 3: Process Orchestration
Exercise 4: Orchestration vs Choreography

Deadline: 17.03.2026, 23:59

Group 1, Team Members:
Michael Schütz, Gianluca Ielpo, Eva Amromin

Contents

Overview . . . 2
Architecture Characteristics . . . 2
Context Map . . . 3
Architecture . . . 3
  Kafka Communication Design . . . 3
    Topics . . . 4
    Consumer Groups . . . 4
    Broker Configuration . . . 4
Process Orchestration . . . 5
  Order Service . . . 5
  Inventory Service . . . 5
  Factory Service . . . 6
  Dashboard Service . . . 6
  Dobot Control . . . 6
  Color Sensor . . . 6
ADRs . . . 8
Contributions . . . 8

Overview

github.com/ielpo/edpo-project-group1

This project implements an event-driven manufacturing scenario for custom furniture orders. The project is called **KAFKEA**, a blend of *Kafka* and *IKEA*. The core objective is to design and validate how distributed services can coordinate reliably when the business flow spans multiple technical boundaries.

At a business level, we want to achieve three outcomes:

- Customers can place furniture orders and receive transparent status updates from start to completion.
- The system can coordinate inventory reservation, manufacturing by the Dobot Magician robots, and completion reporting across services.
- Failure scenarios (service outage, timeout, partial processing) are handled in a controlled way through retries, error paths, and compensation.

At an architecture level, the project demonstrates a hybrid model:

- *Orchestration* for critical process control (order and factory workflows).
- *Choreography* for status propagation and decoupled consumers (event subscribers such as a read-only dashboard).

The implementation is intentionally close to real distributed-system constraints: remote calls can fail, processes can be long-running, and consistency across services is eventual rather than immediate. The project therefore focuses not only on the happy path, but also on resilience patterns and recovery.

Architecture Characteristics

The following architecture characteristics were identified as the most relevant driving qualities for KAFKEA. They reflect the system's core constraints as a distributed, event-driven manufacturing platform where correctness, resilience, and availability are non-negotiable.

Characteristic	Relevance to KAFKEA
Fault Tolerance	The system must continue operating when individual components fail. KAFKEA handles this through BPMN error paths, timeout boundaries, retries, and compensation flows, for example restocking inventory when manufacturing fails.
Data Consistency	State is distributed across Order, Inventory, and Factory services with no shared database. Consistency is eventual and maintained through correlated Kafka events and persisted workflow state rather than synchronous transactions.
Interoperability	KAFKEA integrates heterogeneous technologies: HTTP REST between services, Kafka for async messaging, a BPMN engine

Characteristic	Relevance to KAFKEA
	for process control, a serial port connection to the Dobot robot arm, and a color sensor running on a Raspberry Pi Pico. Each boundary requires a deliberate integration contract.
Reliability	Manufacturing commands and status events must not be lost. KAFKEA relies on Kafka's durability guarantees and BPMN engine persistence to ensure that every order reaches a defined outcome, even across restarts.
Availability	The system must remain accessible to customers during operation. The current single-node Kafka setup is a deliberate tradeoff for development simplicity; production-grade availability would require a multi-broker cluster with higher replication factors.

Context Map

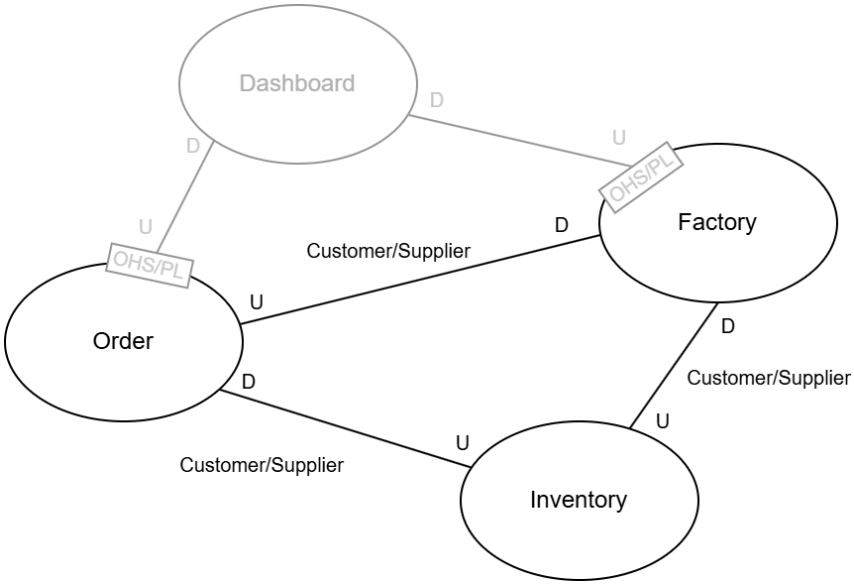

Figure 1: Context map

The context map shows how the bounded contexts in the KAFKEA system relate to one another. At the core, three domain services handle the primary business flow: *order*, *factory*, and *inventory*. Supporting contexts cover visualization (*dashboard*) and hardware integration which is part of the factory (*dobot-control*, *color-sensor*).

Architecture

Kafka Communication Design

Kafka is used for cross-service communication whenever the interaction should be asynchronous and decoupled. Topics are split by intent into commands and events (see ADR 7), and the system runs on a single-node broker with `latest` offset reset across all consumers (see ADR 8).

Topics

Topic	Purpose	Produced by	Consumed by	Notes
order.manufacture.v1	Start manufacturing for an accepted order	Order	Factory	Command topic; keeps factory startup explicit and controlled by the orchestrator
order.complete.v1	Manufacturing completed successfully	Factory	Order, Dashboard	Lets order process continue asynchronously and UI update without tight coupling
info.v1	Informational status events	Order, Factory	Dashboard	User-facing progress updates kept separate from workflow control
error.v1	Failure and timeout status events	Order, Factory	Order, Dashboard	Consolidates failure communication; supports correlation back to running processes

Consumer Groups

Service	Consumer Group	Main Topics
Order	order-service-correlation	order.complete.v1, error.v1
Factory	factory-service	order.manufacture.v1
Dashboard	dashboard	info.v1, error.v1

All consumers are configured with *latest* because this system processes live workflow traffic and does not rely on startup replay from Kafka history. In this setup, replaying backlog can re-emit stale status updates and old command/events. Therefore consumers use latest by default. Process progress is primarily recovered from persisted workflow state and service data stores, with Kafka replay not used as the main recovery mechanism.

Broker Configuration

- Single-node KRaft, one process as broker and controller.
- Advertised listener: `PLAINTEXT://localhost:9092`.
- Default one partition per topic, replication factor 1 (including the internal offsets topic).

Process Orchestration

The process begins when a customer fills out the order form seen in Figure 2. This action represents the start event of the overall order process. Currently, the system supports ordering a single item in a selected colour per order; however, it is designed to be extendable to multiple items.

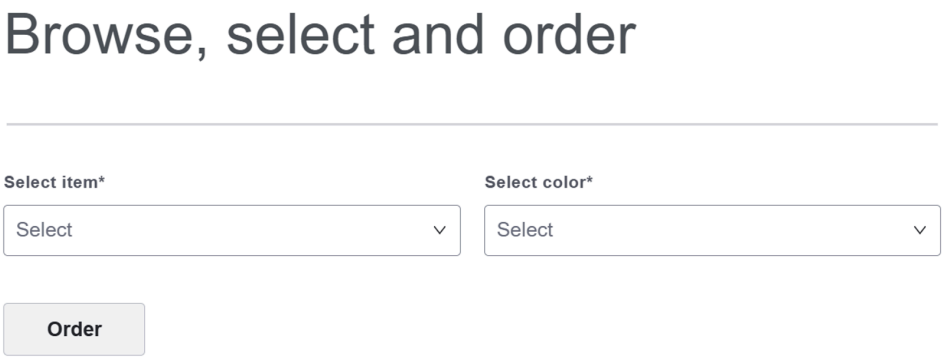

Figure 2: Order form presented to customer

Order Service

github.com/ielpo/edpo-project-group1/services/order

The order service orchestrates the end-to-end business process by coordinating the workflow. As seen in Figure 3, the order service first reserves the required components in inventory via an HTTP call, then sends the manufacturing command, waits for correlated outcome events, enforces timeout handling, and triggers compensation when needed.

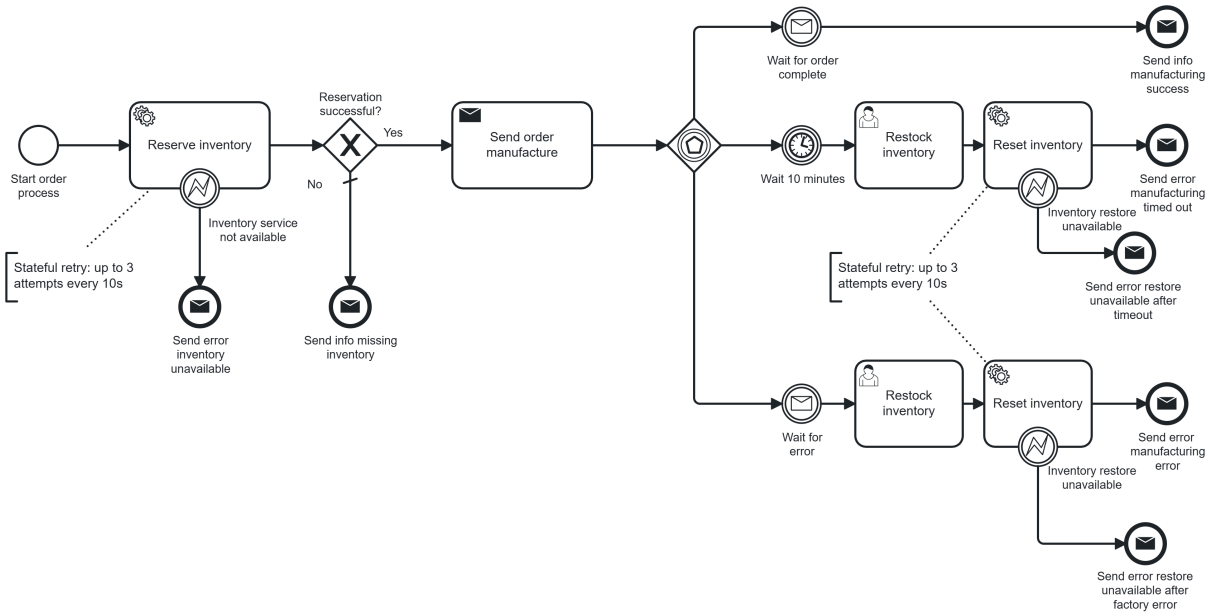

Figure 3: Order process

Inventory Service

github.com/ielpo/edpo-project-group1/services/inventory

The inventory owns stock state and the reservation lifecycle. It is implemented as a separate REST service that exposes its API for reservation, restocking and retrieval operations. If the reservation succeeded, the order service sends a command to the factory to initiate the manufacturing process.

Factory Service

github.com/ielpo/edpo-project-group1/services/factory

The factory process manufactures the requested item and emits events to inform other services about the progress and completion of the order.

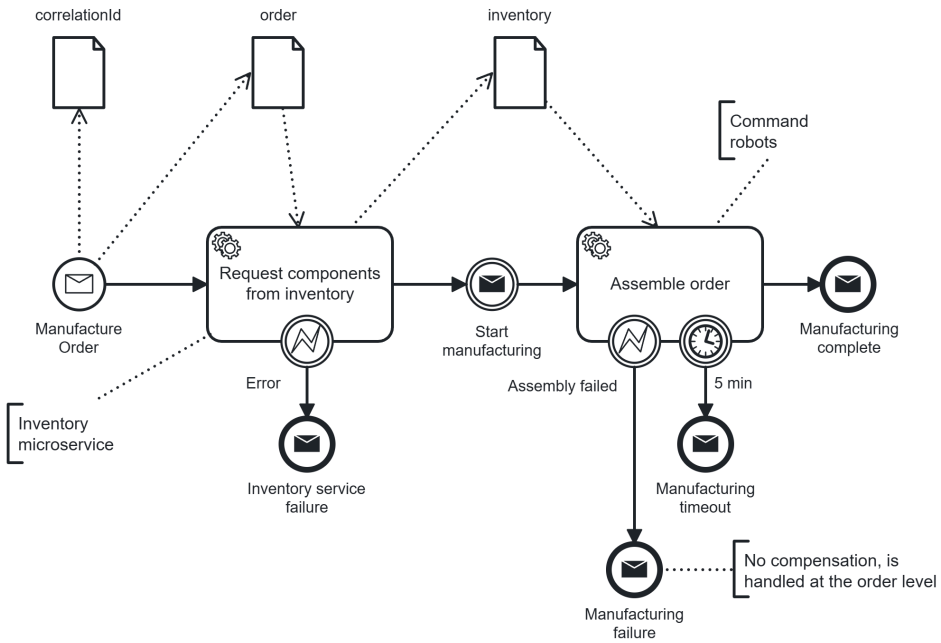

Figure 4: Factory process

Dashboard Service

github.com/ielpo/edpo-project-group1/services/dashboard

The dashboard service is responsible for presenting relevant information to the user. It receives updates from both the order service and the factory process and displays the current status of the order to the customer.

Dobot Control

github.com/ielpo/edpo-project-group1/services/dobot-control

Communicates with the Dobot using the serial port and provides a REST API to send commands to the physical robot arm.

Color Sensor

github.com/ielpo/edpo-project-group1/services/color-sensor

github.com/ielpo/edpo-project-group1/services/color-sensor-fake

Runs on a RaspberryPi Pico and controls and reads out color using an I2C color sensor. A *color-sensor-fake* variant is available as a development/test double.

ADRs

The following ADRs are related to these exercises:

- ADR 0001: Record architecture decisions using ADRs
- ADR 0002: Use orchestration for order/manufacturing and choreography for customer-facing updates.
- ADR 0003: Use Operaton as BPMN engine.
- ADR 0004: Use commands to trigger services and events to report outcomes.
- ADR 0005: Inventory is the source of truth for item color.
- ADR 0006: Dashboard is a separate service for event visualization.
- ADR 0007: Structure of the Kafka topics.
- ADR 0008: Kafka Deployment and Consumer Offset Policy.

Contributions

Person	**Tasks**
Michael	Model processes, Report
Eva	Model processes, ADRs
Gianluca	Model processes, ADRs, Report